REPLACING REPOSITORY EXPLORATION WITH TYPED JEV DECISIONS
CS329Z MIDWAY REPORT

Kristjan Dagur Egilsson **Ómar Ingi Halldórsson** **Rami Ratl Mrad**

1 RESEARCH QUESTIONS

Our project asks whether a structured decision model can replace a narrow component of an existing coding agent at lower total cost while preserving bug-repair performance. Following the instructor's recommendation to obtain mechanical evidence before redesigning an agent harness, we study OpenCode's repository-exploration subagent. Our intervention uses Jev to select relevant source files from deterministic search candidates; the same strong generative model continues to write and test patches. We compare this intervention against the native generative explorer and a search-only replacement. The primary outcomes are independently tested repair success and total inference cost. The midpoint objective is to establish a working substitution and an authentic evaluation pipeline, not to claim calibrated confidence or performance equivalence from a small development sample.

2 ENVIRONMENT & DATA

Existing agent harness. We use the pinned OpenCode 1.19.5 command-line harness and retain its native build agent, `task` delegation tool, `explore` subagent, file tools, and patch-writing loop (OpenCode contributors, 2026). Each rollout starts from an exact upstream repository snapshot. Its issue prompt asks the build agent to delegate initial localization to the existing explorer before editing, then implement a minimal fix and optionally run existing tests. This prompt is identical across conditions. The parent can recover from imperfect localization using its usual read/search tools. It may edit implementation files under `pydicom/`, but not tests or files outside the repository through the configured tool permissions. These permissions are not an operating-system sandbox.

Authentic development tasks. We use all five pydicom issues in the official SWE-bench Lite development split (Jimenez et al., 2024; SWE-bench contributors, 2026): instances ending in 1139, 1256, 1413, 1694, and 901. The dataset revision, task IDs, and upstream base commits were frozen before agent outcomes. We selected this repository for a locally runnable pure-Python environment; this convenience sample is neither representative of SWE-bench nor an independent final test set. The tasks concern person-name iteration, nested JSON bulk-data handling, byte-valued data elements, suppressed JSON conversion errors, and logging configuration. No invented programming tasks enter the corrected pilot.

The dataset supplies issue descriptions, upstream fix patches, added regression-test patches, and fail-to-pass/pass-to-pass test identifiers. Only the issue and original repository are available to the coding agent. The evaluator's added tests, gold fix, and expected outcomes remain outside the agent workspace and Jev state. We do not supply future test feedback to Jev or change the protocol after observing failed repairs. Public benchmark issues may be present in model training data; we make no contamination-free claim.

Independent local evaluation. After each rollout, the evaluator applies the generated implementation patch and the benchmark's test patch to a separate fresh base snapshot. It executes the supplied fail-to-pass and pass-to-pass selections under Python 3.12 with pinned pytest 7.4.4, setuptools 70.3.0, NumPy 1.26.4, and Pillow 11.3.0. Success requires zero test failures/errors/skips, the expected minimum collection, successful agent termination, and no forbidden tracked-file changes. Every environment was checked before paid agent runs: its broken base fails assertions and its published upstream fix passes. The five controls execute 41, 23, 304, 27, and 5 cases respectively. These are test cases within five issues, not hundreds of independent tasks.

Historical tests need pytest 7's nose-style setup, and three truncated test identifiers in instance 1413 are expanded to their whole test functions rather than dropped. Because local pytest differs from SWE-bench's containerized evaluation, we report *local benchmark-regression success*, not an official resolved score; official container reproduction remains required for the final study.

File-selection evaluation on 225 development bugs. End-to-end success on five issues cannot isolate the one decision Jev makes, and the parent agent can recover from a poor file choice. We therefore also evaluate that decision directly on every instance of the full SWE-bench development split (Jimenez et al., 2024): 225 real issues from six Python repositories (pydicom, sqlfluff, pvlib, astroid, pyvista, marshmallow). Each instance's *gold files* are the implementation files its official fix modifies; they are used only by the scorer. For each issue we build the same 12 candidates as the agent, from the issue text at the base commit, and compare the three files chosen by search alone and by Jev. The primary metric is *hit@3*, the share of issues with at least one gold file among the three chosen. We also report candidate recall (a gold file among the 12, the ceiling for any selector) and hit@3 restricted to issues where retrieval found a gold file. Jev and search are compared on the same issues with a paired bootstrap interval and an exact sign test. The protocol was committed before any decision was requested. The SWE-bench test split and SWE-bench Verified are reserved for held-out final evaluation.

3 METHODS

One internal substitution. The build agent uses `openai/gpt-6.1-sol` in every condition. A thin OpenAI-compatible provider adapter exposes distinct build and explore URLs. OpenCode's native `task` tool still creates an explorer session and returns its result to the parent. In the native condition, both provider URLs forward to the same strong model. In the Jev condition, only an explore-provider request is replaced with deterministic retrieval, one typed Jev request, and a formatted list of selected file paths/snippets. There is no additional generative prescreen or newly written agent execution loop.

Candidate construction and Jev decisions. We index Python implementation files, excluding the repository's tests. A deterministic BM25-style search ranks files using the parent agent's actual exploration request. The top 12 candidates contain relative paths and original source snippets: the most query-matching 16-line window in each file, capped at 1,800 characters. This construction uses no gold labels. Jev 1.14 receives the request and candidates and answers one Noul file-relevance question per candidate in a single API call (OpenRouter, 2026). We validate every returned probability, rank files by score, and return the top three paths/snippets to the native explorer session. Thus, Jev determines a concrete internal context-selection decision; it neither generates a patch nor selects the coding model at task launch. The offline file-selection study reuses this retrieval and the identical Jev question, generalized from `pydicom/` to each repository's package directory; it reproduces the pilot's ten archived candidate lists exactly. Its one difference is the query: the issue text, rather than the parent agent's exploration request.

An API error, malformed response, or invalid score falls back to the ordinary generative explorer, counting both the attempted decision and fallback costs. This pilot ranks candidates without a confidence threshold. Noul relevance scores are not calibrated probabilities of patch correctness. A future confidence-based escalation policy requires independent development/calibration data; it cannot be justified by selecting a threshold on these observed failures.

Comparisons and fixed limits. We run three conditions on the same five issues, one rollout each: (1) native generative exploration; (2) retrieval plus Jev selection; and (3) search-only, returning the first three retrieval candidates without Jev. The search-only control asks whether any improvement requires the decision model. The native explorer has whole-repository tool access, whereas both replacements use bounded candidates; this is a component replacement comparison, not identical-context ranker isolation. Parent trajectories are stochastic, and the parent may recover using later searches. A final study should also compare a generative ranker receiving the same candidates and use repeated rollouts where affordable.

All conditions share the issue prompt, strong writer, tools, evaluator, 16 parent steps, 6 native explorer steps, at most 32 model requests, 4,096 output tokens/request, and 480 seconds/run. We

Table 1: Real pydicom development pilot; one rollout per issue/condition. Cost includes all captured model calls. Time is observed agent wall time through the common local adapter, excluding separate evaluator time. Local regression outcomes are not official Docker SWE-bench scores.

Exploration condition	Repairs passed	Mean cost (USD)	Mean time (s)
Native generative explorer	2/5	0.110382	37.64
Jev file selection	2/5	0.072762	25.93
Search only	2/5	0.062014	24.90

Table 2: File selection on the SWE-bench development split (one Jev decision per issue). Recall@12: a gold file is among the 12 candidates. Hit@3: a gold file is among the 3 chosen. W/L: issues where only Jev, or only search, chose a gold file.

Repository	Issues	Recall@12 (%)	Search hit@3 (%)	Jev hit@3 (%)	W/L
marshmallow	9	100.0	100.0	100.0	0/0
pvlib	63	95.2	88.9	93.7	4/1
pydicom	56	92.9	67.9	85.7	11/1
astroid	31	90.3	61.3	87.1	8/0
pyvista	16	75.0	43.8	68.8	5/1
sqlfluff	50	78.0	58.0	76.0	10/1
All	225	88.9	70.2	85.3	38/4

execute the native baseline before activating Jev. Task outcomes are paired by issue, but the small sample does not support a noninferiority claim. We report every declared rollout, including unsuccessful repairs.

Accounting and artifacts. The adapter records every model response and its billed usage, and a durable ledger enforces caps of $2 per rollout and $15 across the pilot. The coding process never receives the real API key. Each immutable run stores its source/protocol hash, event trace, request/response receipts, file decisions, patch, test log, and result; all 15 runs share one source hash and all bills were captured.

4 Summary of Progress

Working prototype and measured comparison. The internal substitution is implemented and exercised on real repository repairs. Table 1 summarizes all 15 development rollouts. Both the native and Jev conditions solved 2/5 issues. Mean total cost changed from $0.110382 to $0.072762, a 34.1% reduction for this sample. Mean agent wall time changed from 37.64 to 25.93 seconds. The five Jev runs used five real decision calls and zero fallback explorer generations, replacing 23 native explorer generation calls. These measurements establish mechanical feasibility; matching observed success on five development issues does not establish equivalent performance.

What the control and failures tell us. The search-only condition solved 2/5 issues with mean cost $0.062014. On cost, this control suggests that bypassing the iterative explorer explains much of the saving. On end-to-end success the three conditions are indistinguishable on five issues, but the file-selection study below shows that Jev does choose files better than search alone. For example, task 1139's patches implement iteration/containment but fail the benchmark's direct `next(person_name)` requirement. Task 1413's repairs do not handle every byte-valued data-element variant. Task 901 changes logging configuration but fails the benchmark's handler/debug compatibility expectations. These are observable repair failures, not evaluator infrastructure errors: the corresponding gold fixes pass. Imperfect localization can also be hidden by the strong parent's later recovery, so file-selection quality and end-to-end success should be analyzed separately in a broader study.

Jev chooses files better than search. On all 225 development issues (Table 2), Jev placed a gold file among its three choices for 85.3% of issues, compared with 70.2% for search alone and 30.8% expected from a random choice among the same candidates. The paired difference is 15.1 percentage points (95% bootstrap interval 9.8 to 20.4). Jev was right where search was wrong on 38 issues and the reverse on 4 (sign test $p < 0.001$). When retrieval found a gold file (200 issues), Jev chose one 96.0% of the time, versus 79.0% for search; the remaining errors are mostly retrieval misses, so candidate recall (88.9%) now bounds the selector. The gain holds in every repository except marshmallow, where both are perfect. The study cost $0.0491. In the 15 pilot rollouts the explorer's report contained a gold file in 5/5 native, 3/5 Jev and 3/5 search runs, yet all 15 final patches edited the gold file: the strong parent recovered from every localization miss, which is why end-to-end success did not separate the conditions.

Longer shortlists. A separately frozen sweep repeated the study with 24 and 48 candidates. Jev's hit@3 rose from 85.3% to 90.2% and 91.1%: of the 25 issues whose gold file missed the 12 candidates, Jev recovered 13 at 24 and 15 at 48 (two are files created by the fix), while keeping 189 of the 192 issues it already solved at 12. The 24-candidate gain over 12 is 4.9 points (95% interval 1.3 to 8.4; 14 wins, 3 losses, $p = 0.013$). Median decision latency grew only from 0.29 to 0.34 s, but at 48 candidates 44 of 225 decisions took about 8 s, against 0 at 24; decisions cost $0.22, $0.39 and $0.69 per thousand. A 24-candidate shortlist therefore captures most of the benefit without the latency tail.

Completed work and next steps. Environment controls, the Jev integration, all three pilot conditions, metered trace artifacts, and offline checks for malformed decisions, gold/test leakage, fallback, and budget denial are complete (15 offline tests pass). Both studies can be recomputed offline from their archives. The pilot cost $1.225791 and the file-selection studies $0.29; no Tinker credits were used.

For the final study, we will reproduce with the official evaluator, broaden repositories and development tasks, examine retrieval/selection errors and parent recovery, and freeze independent evaluation settings before inspecting final outcomes. We will report paired uncertainty, costs, latency, and failure categories rather than extrapolating this sample: confidence intervals are not meaningful for five single rollouts (2/5 has a 95% interval of roughly 12% to 77%), so the final study will use repeated rollouts on more issues and paired bootstrap intervals per condition, as in the file-selection study. The next end-to-end runs extend the pilot to the 51 further pydicom development issues under four conditions (native, Jev, search, and an oracle given the gold files as an upper bound), with wall time as a primary outcome alongside repair success and cost. The original BANKING77/CLINC150 intent-decision and tool-using escalation proposal remains a deferred option. Training with the team's separate Tinker accounts is conditional on a demonstrated need and sufficient separately sourced data; it is not a prerequisite for this experiment.

Safety considerations. We use public software issues and repository fixtures, not clinical patient data or deployed medical systems. Generated patches are research artifacts and are not merged upstream or applied in production. Credentials are excluded from published traces. Future evaluation environments should use container/process isolation in addition to tool-level permissions, retain upstream license notices, and record actual resource cost.

REFERENCES

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*. https://arxiv.org/abs/2310.06770.

OpenCode contributors. OpenCode 1.19.5, native task/explore implementation. Software source. Accessed November 6, 2026. https://github.com/anomalyco/opencode/blob/v1.19.5/packages/opencode/src/tool/task.ts.

OpenRouter. Jev documentation: using the TypeSafe decision model on OpenRouter. API documentation. Accessed November 6, 2026. https://openrouter.ai/docs/guides/community/jev.

SWE-bench contributors. SWE-bench Lite dataset, development split. Revision 6ec7bb89b9342f664a54a6e0a6ea6501d3437cc2. Accessed November 6, 2026. https://huggingface.co/datasets/princeton-nlp/SWE-bench_Lite.

AI-USE DISCLOSURE STATEMENT

Rami. OpenAI Codex assisted with interpreting the instructor feedback, inspecting source and documentation, implementing the provider adapter and evaluator, executing the model experiments, and drafting this report. **Ómar.** Anthropic's Claude Code assisted with reviewing the repository and this report, editing the report for the required format and this disclosure, preparing the reproduction instructions used for the independent teammate check, and designing, implementing and running the offline file-selection study (its protocol, script, Jev requests and analysis). **Kristjan.** [TO FILL IN: Kristjan's use of AI tools, or "none".] **Within the evaluated system.** Jev 1.14 made file-relevance decisions within the evaluated agent, and GPT-6.1-Sol produced the experiment's patches and native exploration responses. Benchmark issues, upstream fixes, and regression tests came from SWE-bench and pydicom rather than being generated for this project. The authors reviewed the reported results against the preserved run artifacts and take full responsibility for this report's contents.